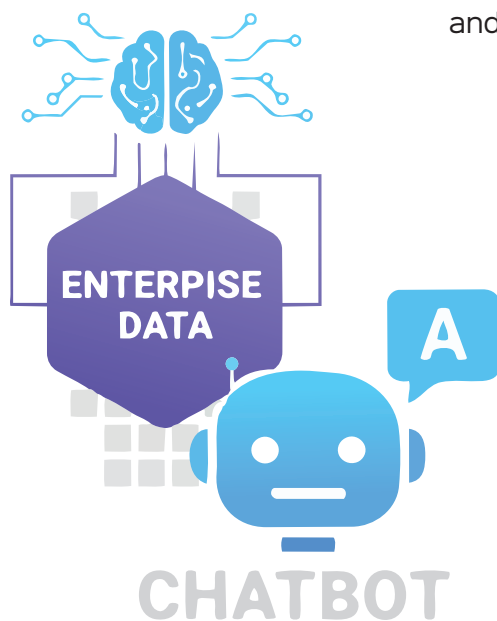


Building a GenAI Chatbot for Enterprise Data

This step-by-step implementation of a genAI-based enterprise chatbot uses various open source frameworks and Python packages.



With significant advancements in natural language processing (NLP) techniques, large language models (LLMs) can today understand, interpret, and generate human language with remarkable accuracy. This has led to organisations leveraging the power of generative AI (genAI) in areas like answering user queries, chatbots for internal tech support, handling customer interactions, etc. GenAI has helped make these interactions seamless, with precise and helpful answers significantly improving the user experience.

Regular genAI models are typically trained on extensive public data for general-purpose tasks and possess general knowledge. In contrast, enterprise data is often private, stored in internal data sources, and cannot be made public. When organisations integrate standard knowledge models into their internal support applications such as enterprise search or QA chatbots, the responses are generic. This may not adequately assist users who need to access private enterprise data to perform their tasks. For instance, consider a support engineer at a company using a standard knowledge LLM to address a user query about the company's product or to gain more insight into the product's features. A regular LLM

trained solely on public data may not provide the necessary information. Its responses could potentially include data about similar products from competitors. It might even generate inaccurate information, often referred to as 'hallucination', which is a prevalent concern with the use of LLMs.

Organisations need solutions that enable the integration of genAI with enterprise data. These must enhance the capabilities of search engines and Q&A chatbots, providing more precise and insightful answers to user queries and reducing the risk of misinformation. Let's look at solutions that address this requirement.

Retrieval augmented generation (RAG)

Retrieval augmented generation (RAG) represents an innovative approach that combines generative AI with enterprise data. RAG effectively bridges the gap between general knowledge models and specific organisational requirements by incorporating additional internal data. The RAG solution is popular today as it can integrate models with proprietary enterprise data. At a high level, RAG involves the following steps:

- Vector embeddings are created for enterprise data present in private data stores and stored in vector stores for querying.
- When a user sends a prompt to the model, an embedding is generated for the query.
- This embedding is matched against the vector store, and data that semantically matches the query embedding is retrieved.
- The user prompt is supplemented with the retrieved data, and both are sent to the model within the context.
- The model processes the prompt and generates the final response using additional contextual data.

The key benefit of the RAG approach is that the model is provided with knowledge of enterprise private data. It can generate responses based on enterprise data without needing any fine tuning or additional training. Since data is private and safe, this addresses one of the major concerns of enterprises. RAG also addresses the issue of hallucinations where models